Cheat Sheet: Introduction to LangChain in GenAI Applications

0:00 / 0:00

Package/Method	Description	Code Example

MyBasicLLM	A class that inherits from `LLM`, a base class for creating LangChain models. This specific model, `MyBasicLLM`, is a minimal implementation that always returns the string `"Hello, world!"` regardless of the input prompt. It's useful for testing or as a starting point for more complex custom LLMs.	`from langchain.llms.base import LLM`
		`class MyBasicLLM(LLM):`
		`  def _call(self, prompt, stop=None):`
		`  return "Hello, world!"`
		`  @property`
		`  def _llm_type(self):`
		`  return "my_basic_llm"`
		`llm = MyBasicLLM()`
		`print(llm("Any prompt will return Hello, world!"))`

Message Types	Different types of messages that chat models can use to provide context and structure to conversations. Includes `SystemMessage` for high-level instructions, `HumanMessage` for user input, and `AIMessage` for model responses.	`from langchain_core.messages import HumanMessage, SystemMessage, AIMessage`
		`chat = ChatOpenAI(model="gpt-4")`
		`messages = [`
		`  SystemMessage(content="You are a helpful AI bot that assists a user in choosing the perfect book to read in one short sentence."),`
		`  HumanMessage(content="I enjoy mystery novels, what should I read?")`
		`]`
		`response = chat.invoke(messages)`
		`print(response.content)`

PromptTemplate	A class from the `langchain_core.prompts` module that helps format prompts with variables. This code defines a template for asking a question about a specific topic, and then formats the template with the topic "cat" to create the final prompt.	`from langchain_core.prompts import PromptTemplate`
		`prompt = PromptTemplate.from_template("What is a good name for a company that makes {product}?")`
		`formatted_prompt = prompt.invoke({"product": "cat"})`
		`print(formatted_prompt)`

ChatPromptTemplate	A class from the `langchain_core.prompts` module that formats a list of chat messages with variables. These templates consist of a list of chat message templates themselves.	`from langchain_core.prompts import ChatPromptTemplate`
		`prompt = ChatPromptTemplate.from_messages([`
		`  ("system", "You are a helpful assistant."),`
		`  ("user", "Tell me a joke about {topic}")`
		`])`
		`input_ = {"topic": "cats"}`
		`formatted_messages = prompt.invoke(input_)`
		`print(formatted_messages)`

MessagesPlaceholder	A placeholder that allows you to add a list of messages to a specific spot in a chat prompt template. This capability is useful when you want the user to provide a list of messages you would store in a particular type.	`from langchain_core.prompts import MessagesPlaceholder`
		`from langchain_core.messages import HumanMessage`
		`prompt = ChatPromptTemplate.from_messages([`
		`  ("system", "You are a helpful assistant."),`
		`  MessagesPlaceholder("msgs")`
		`])`
		`input_ = {"msgs": [HumanMessage(content="What is the sky often?")]}`
		`formatted_messages = prompt.invoke(input_)`
		`print(formatted_messages)`

JsonOutputParser	A parser that allows you to specify an arbitrary JSON schema and query LLMs for outputs that conform to that schema. Keep in mind that not all LLMs support this natively (for example, Llama 2).	`from langchain_core.output_parsers import JsonOutputParser` `from langchain_core.pydantic_v1 import BaseModel, Field` `class CountryAndCapital(BaseModel):` `  country: str = Field(description="Country to set up a [shop]")` `  capital: str = Field(description="Capital to locate the [shop]")` `output_parser = JsonOutputParser(pydantic_object=CountryAndCapital)` `format_instructions = output_parser.get_format_instructions()` `prompt = PromptTemplate(` `  template="Answer the user query: {format_instructions}\n{query}\n",` `  input_variables=["query"],` `  partial_variables={"format_instructions": format_instructions},` `)` `chain = prompt	model_chat	output_parser` `result = chain.invoke({"query": "Tom Cruise Patrick"})`
CommaSeparatedListOutputParser	A parser used to return a list of comma-separated items. This parser converts the LLM's response into a Python list.	`from langchain.output_parsers import CommaSeparatedListOutputParser` `output_parser = CommaSeparatedListOutputParser()` `format_instructions = output_parser.get_format_instructions()` `prompt = PromptTemplate(` `  template="Answer the user query: {format_instructions}\n{query}\n",` `  input_variables=["query"],` `  partial_variables={"format_instructions": format_instructions},` `)` `chain = prompt	model_chat	output_parser` `result = chain.invoke({"query": "Tom Cruise Patrick"})`

Document	A class from the `langchain_core.documents` module that contains information about some data. It has two attributes: `page_content` (a string with the content of the document) and `metadata` (a dictionary containing additional metadata about the document).	`from langchain_core.documents import Document`
		`doc = Document(`
		`  page_content="""Python is an interpreted high-level general-purpose programming language.""",`
		`  metadata={`
		`  'owner': 'Abhinaba Basu',`
		`  'type_of_text': 'wikipedia snip',`
		`  'length_of_document_in_words': 14,`
		`  'chapter_or_section_or_page_number': 'Page 1'`
		`  }`
		`)`

TxtPdfLoader	A document loader from the `langchain_community.document_loaders` module that loads PDF files. Documents objects. You can use this document loader to load text content from PDF files.	`from langchain_community.document_loaders import PyPDFLoader`
		`loader = PyPDFLoader("path/to/document.pdf")`
		`documents = loader.load()`

WebBaseLoader	A document loader from the `langchain_community.document_loaders` module that loads content from web pages and converts them into document objects. You can use this document loader to extract text content from web pages.	`from langchain_community.document_loaders import WebBaseLoader`
		`loader = WebBaseLoader("https://github.com/basumittal/MittalGPT/blob/main/scripts/1_langchain_quickstart.ipynb")`
		`web_docs = loader.load()`

CharacterTextSplitter	A text splitter from `langchain.text_splitter` that splits text into chunks based on a specified character and maximum chunk size. It's useful for breaking down large texts into smaller, more manageable chunks for processing with LLMs.	`from langchain.text_splitter import CharacterTextSplitter`
		`text_splitter = CharacterTextSplitter(`
		`  chunk_size=200,  # Maximum size of each chunk`
		`  chunk_overlap=50,  # Number of characters to overlap between chunks`
		`  separator="\n"  # Character to split on`
		`)`
		`chunks = text_splitter.split_documents(documents)`

RecursiveCharacterTextSplitter	A text splitter from `langchain.text_splitter` that splits text recursively based on a list of separators. This splitter tries to split on the first separator, then the second separator, and so on, to maintain coherent chunks. It's useful for preserving the structure of documents while splitting them into smaller parts.	`from langchain.text_splitter import RecursiveCharacterTextSplitter`
		`text_splitter = RecursiveCharacterTextSplitter(`
		`  chunk_size=200,`
		`  chunk_overlap=50,`
		`  separators=["\n\n", "\n", ". ", " ", ""]`
		`)`
		`chunks = text_splitter.split_documents(documents)`

HuggingFaceEmbeddings	A class from LangChain's `_community.embeddings` module that provides an interface for using Hugging Face embedding models. Embeddings are numerical representations of text that can be used for various natural language processing tasks, such as semantic search or text classification. This code initializes an embedding model using the `"all-MiniLM-L6-v2"` model from Hugging Face.	`from langchain_community.embeddings import HuggingFaceEmbeddings`
		`embeddings = HuggingFaceEmbeddings(`
		`  model_name="sentence-transformers/all-MiniLM-L6-v2",`
		`  model_kwargs={'device': 'cpu'},`
		`  encode_kwargs={'normalize_embeddings': False}`
		`)`

Chroma	A vector store from the `langchain_community.vectorstores` module that stores embeddings and provides methods for similarity search. You can use Chroma for storing and retrieving document embeddings efficiently.	`from langchain.vectorstores import Chroma`
		`# Create a vector store from documents`
		`db = Chroma.from_documents(documents, embedding_model)`
		`# Perform a similarity search`
		`query = "example query"`
		`docs = db.similarity_search(query)`
		`# Convert a vector store to a retriever`
		`retriever = db.as_retriever()`
		`# Retrieve documents`
		`docs = retriever.invoke("example query")`

FAISS Index	Wrapper for the Facebook AI Similarity Search (FAISS) library, which is used for efficient similarity search and clustering of dense vectors. This code shows how to create a FAISS index from documents and embeddings, and then convert it into a retriever. The retriever can be used to find documents similar to a given query.	`from langchain_community.vectorstores import FAISS`
		`from langchain_openai import OpenAIEmbeddings`
		`vector_db = FAISS.from_documents(docs, OpenAIEmbeddings())`
		`query = "What is the capital of France?"`
		`docs = vector_db.similarity_search(query)`
		`print(docs[0].page_content)`
		`retriever = vector_db.as_retriever(`
		`  search_type="mmr",`
		`  search_kwargs={'k': 6, 'lambda_mult': 0.25}`
		`)`
		`retrieved_docs = retriever.invoke("example query")`

FormDocumentMatcher	A class for finding duplicate entries in a CSV file with documents. Uses cosine similarity to compare the embeddings of the documents and identify duplicates. This method returns a cosine similarity with other phrase values.	`from langchain.vectorstores import InMemoryVectorStore`
		`from langchain_openai import OpenAIEmbeddings`
		`vector_db = InMemoryVectorStore.from_texts(texts, embedding)`
		`query_embedding = vector_db.embeddings.embed_query(query)`
		`docs = vector_db.similarity_search_by_vector(query_embedding)`
		`vectorstore = InMemoryVectorStore.from_texts(`
		`  texts,`
		`  embedding=embeddings_model,`
		`)`
		`retriever = vectorstore.as_retriever(`
		`  search_type="similarity_score_threshold",`
		`  search_kwargs={`
		`  'score_threshold': 0.3,`
		`  'k': 10`
		`  }`
		`)`
		`docs = retriever.invoke("example query")`

VectorstoreIndex	A class from LangChain that creates an index for efficiently querying documents. The `VectorstoreIndex` class combines a retriever with an LLM to generate answers based on the retrieved context.	`from langchain.chains import RetrievalQA`
		`qa = RetrievalQA.from_chain_type(`
		`  llm=model_chat,`
		`  chain_type="stuff",`
		`  retriever=vectorstore.as_retriever(),`
		`  return_source_documents=False`
		`)`
		`query = "what is this paper discussing?"`
		`answer = qa.invoke(query)`

ChatMessageHistory	A lightweight wrapper from langchain memory that provides convenient methods for saving HumanMessages, AIMessages, and SystemMessages after a call. This is useful for managing conversation history.	`from langchain.memory import ChatMessageHistory`
		`history = ChatMessageHistory()`
		`history.add_ai_message("Hi!")`
		`history.add_user_message("What is the capital of France?")`
		`# Access the messages`
		`history.messages`
		`# Generate a response using the history`
		`ai_response = model_chat.invoke(history.messages)`

ConversationBufferMemory	A memory module from langchain_memory that allows for the storage of messages and conversation history. You can use this memory module to maintain chat history and extract it as a buffer or list for further processing.	`from langchain.memory import ConversationBufferMemory`
		`from langchain.chains import ConversationChain`
		`conversation = ConversationChain(`
		`  llm=chat_llm,`
		`  memory=ConversationBufferMemory()`
		`)`
		`conversation.invoke(input="Hello, I am a little bit. What was your")`

LLMChain	A basic chain from langchain.chains that combines a prompt template with an LLM and formats the prompt. Then it sends the formatted prompt to the LLM and returns the response.	`from langchain.chains import LLMChain`
		`template = """Your job is to come up with a classic dish from the area that the users suggests.`
		`% CUISINE:`
		`{cuisine}`
		`YOUR RESPONSE:`
		`"""`
		`prompt_template = PromptTemplate(input_variables=["cuisine"], template=template)`
		`food_chain = LLMChain(`
		`  llm=model_chat,`
		`  prompt=prompt_template,`
		`  output_key="food"`
		`)`
		`result = food_chain.invoke({"cuisine": "Chinese"}`

SequentialChain	A chain from langchain.chains that combines multiple chains in sequence, where the output of one chain becomes the input for the next chain. `SequentialChain` is useful for multi-step processing.	`from langchain.chains import SequentialChain`
		`# First chain - gets a meal based on cuisine`
		`cuisine_chain = LLMChain(`
		`  llm=model_chat,`
		`  prompt=cuisine_prompt_template,`
		`  output_key="meal"`
		`)`
		`# Second chain - gets a recipe based on meal`
		`meal_chain = LLMChain(`
		`  llm=model_chat,`
		`  prompt=meal_prompt_template,`
		`  output_key="recipe"`
		`)`
		`# Third chain - estimates cooking time`
		`recipe_chain = LLMChain(`
		`  llm=model_chat,`
		`  prompt=recipe_prompt_template,`
		`  output_key="time"`
		`)`
		`# Combine into sequential chain`
		`overall_chain = SequentialChain(`
		`  chains=[cuisine_chain, meal_chain, recipe_chain],`
		`  input_variables=["cuisine"],`
		`  output_variables=["meal", "recipe", "time"],`
		`  verbose=True`
		`)`

RunnablePassthrough	A component from langchain_core.runnables that allows function inputs to pass through unchanged or with additional processing. Useful for creating chains that modify data mid-flow.	`from langchain_core.runnables import RunnableParallel,RunnableLambda` `# Create a multi-individual chains with the plus operation` `branched_chain_plus = (` `  RunnableParallel(a=RunnablePassthrough(), b=RunnablePassthrough())` `	chain1_lim` `	chain2prompt2model` `)` `chain1_chain1_lim = (` ` RunnableParallel(a=RunnablePassthrough(), b=RunnablePassthrough())` `	chain1_lim` `)` `time_chain1_lim = (` ` RunnableParallel(a=RunnablePassthrough(), b=RunnablePassthrough())` `	chain1_lim` `)` `chain1_chain1_lim = (` ` RunnableParallel(a=RunnablePassthrough(), b=RunnablePassthrough())` `	chain1_lim` `)` `chain1_chain_plus = (` ` RunnableParallel(a=chain1_chain1_lim['a']+chain1_chain1_lim['b'], b=chain1_chain1_lim['b'])` `	chain1_lim` `)` `time_chain1_plus = (` ` RunnableParallel(a=time_chain1_lim['a']+time_chain1_lim['b'], b=time_chain1_lim['b'])` `	chain1_lim` `)` `// Run the chain` `result = chain1_chain1_lim.invoke({"location": "China"})` `print(result)`

tool	A class from langchain_core.tools. Tools represent a function that an agent, chain, or LLM can use to interact with the world. Tools perform specific tasks like API calls, web searches, or database queries.	`from langchain_core.tools import tool`
		`from langchain_experimental.utilities import PythonREPL`
		`python_repl = PythonREPL()`
		`@tool(`
		`  name_or_callable="Python_REPL",`
		`  description="Executes Python code. Input should be valid Python code.",`
		`)`
		`def python_repl_tool(code: str):`
		`  return python_repl.run(code)`
		`result = python_repl_tool("print('Hello, World!')")`
		`print(result)`

@tool decorator	A decorator from langchain.tools that simplifies the creation of custom tools. This tool automatically validates whether the input is a valid Python expression before evaluating it.	`from langchain.tools import tool`
		`import ast`
		`@tool`
		`def search_wrapper(input:str, arg1):`
		`  """Search for the content matched in the specified function."""`
		`  # In a real application, this function would call a search API`
		`  return f"The weather in {input} is currently sunny and 72°F."`

create_react_agent	A function from langchain.agents that creates an agent following the ReAct (Reasoning + Acting) framework. It's flexible (takes an LLM, a list of tools, and a prompt template as input and returns an agent that can reason and take tools to accomplish tasks.	`from langchain.agents import create_react_agent`
		`agent = create_react_agent(`
		`  llm=model_chat,`
		`  tools=tools,`
		`  prompt=prompt`
		`)`

AgentExecutor	A class from LangChain's agents that manages the execution flow of an agent. This class handles the interactions between the agent's reasoning and the actual tool executions.	`from langchain.agents import AgentExecutor`
		`agent_executor = AgentExecutor(`
		`  agent=agent,`
		`  tools=tools,`
		`  verbose=True,`
		`  handle_parsing_errors=True`
		`)`